IOT Assignment

IoT-Based Smart Room Monitoring and Automation (Option 1)

Date: 03/11/2026

Aim

To design and implement an ESP32-based IoT system that monitors room temperature, humidity and light level, publishes the data over Wi-Fi using MQTT to a web dashboard, and automatically controls a fan and a light (with an alarm for critical temperature) using threshold-based decisions.

Contents

1. Introduction 2

2. System Design 2

3. Implementation 4

4. Results 5

5. Limitations and Improvements 6

6. Conclusion 6

References 6

1. Introduction

Fans and lights in rooms are often left running when they are not needed, or switched on too late. A smart room controller measures the conditions in the room and acts on its own, while also letting a user monitor and override it remotely. This project implements such a controller on an ESP32, following the sensor → controller → IoT platform → dashboard architecture given in the assignment.

Table 1.1: *Problem statement*

Aspect	This project
What is sensed	Air temperature and relative humidity (DHT22), illuminance (LDR module)
Decisions made on the device	Fan on/off, light on/off, critical-temperature alarm, every 2 s
Actuators / alerts	2 relay modules (fan, light), buzzer, OLED, MQTT alert messages
Connectivity and protocol	Wi-Fi (802.11 b/g/n), MQTT over TCP (device) and over WebSockets (browser)
Monitoring	Web dashboard: live values, trend chart, event log, manual control, thresholds
Implementation platform	Wokwi ESP32 simulator; the same firmware runs on a physical ESP32 DevKit

Requirements coverage:

- ≥ **2 sensors:** DHT22 (temperature + humidity) and LDR (light).
- **Communication and protocol:** Wi-Fi + MQTT (public HiveMQ broker).
- **Dashboard:** `dashboard.html`, connected to the same broker.
- **Threshold-based actuation and alerts:** fan and light relays with hysteresis, buzzer + alert topic.

2. System Design

2.1 Architecture

The ESP32 samples the sensors, applies the decision rules locally and drives the actuators, so the room is still controlled when the network is down. Every 2 s it publishes a JSON telemetry message to the MQTT broker. The browser dashboard subscribes to the same topics over secure WebSockets and publishes commands back to the device.

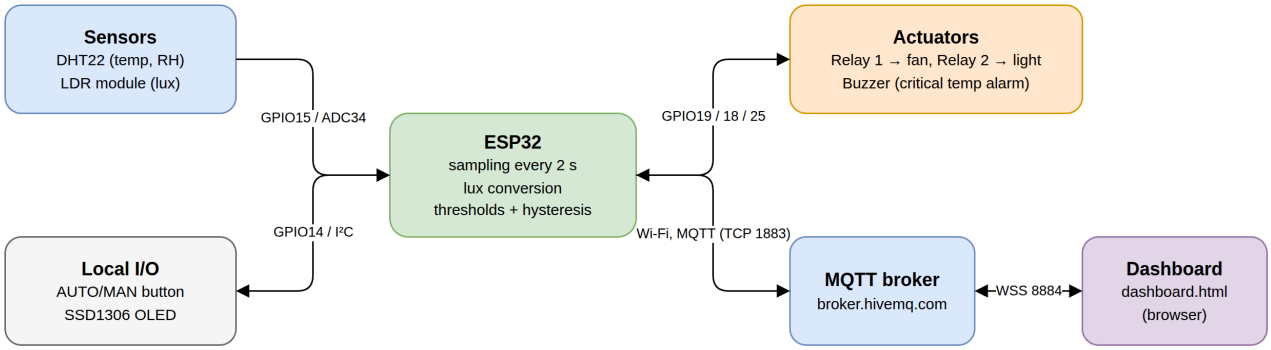


Fig. 2.1: *System architecture*

2.2 Hardware and Circuit

The circuit was built in the Wokwi simulator (`diagram.json`). The two relay contacts switch LEDs that stand in for the fan and the lamp; in a real installation the same contacts would switch the mains loads.

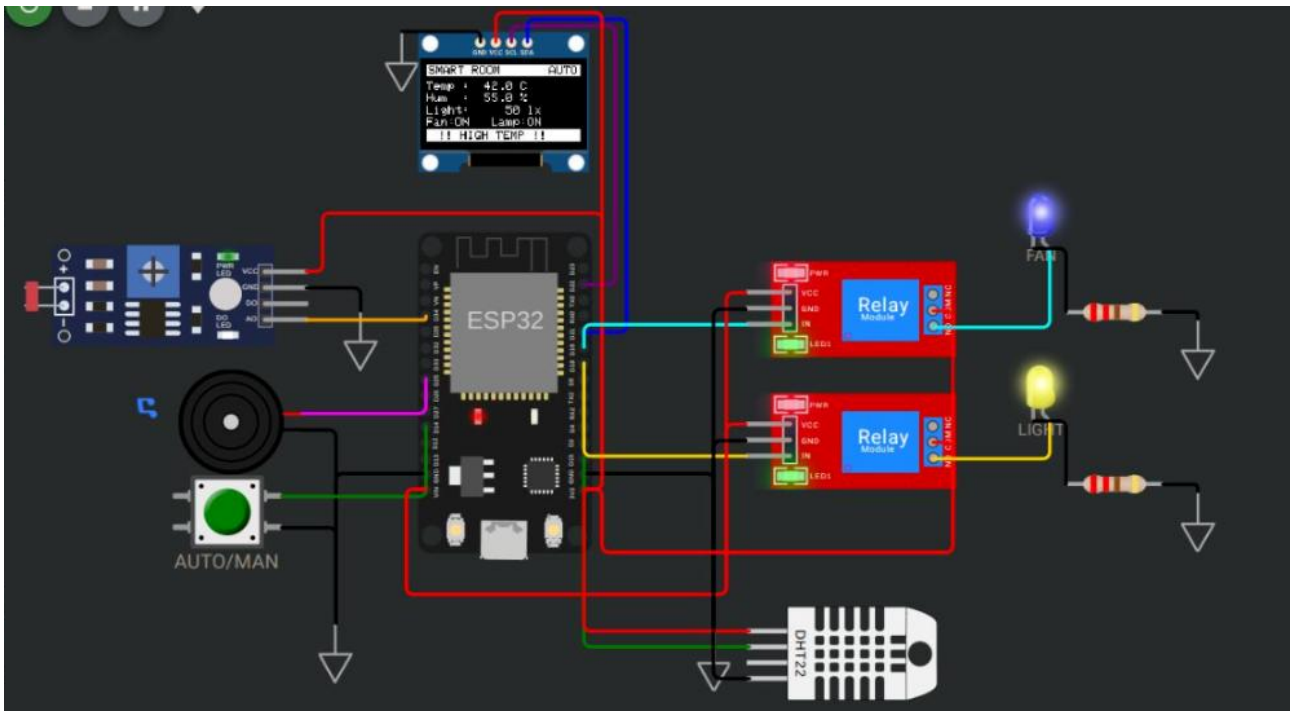


Fig. 2.2: *Circuit in the Wokwi simulator during the alarm test (42 °C, 50 lx): both relays on, OLED shows the alarm*

Table 2.1: *Hardware connections*

Component	Signal pins	ESP32 connection
DHT22 temperature + humidity	DATA, VCC, GND	GPIO15, 3V3, GND
LDR photoresistor module	AO, VCC, GND	GPIO34 (ADC1), 3V3, GND
Relay module 1 (fan)	IN, VCC, GND	GPIO19, VIN (5 V), GND
Relay module 2 (light)	IN, VCC, GND	GPIO18, VIN (5 V), GND
Loads (simulated)	relay COM → 3V3, NO → LED + 220 Ω → GND	blue LED = fan, yellow LED = lamp
Buzzer	+, –	GPIO25, GND
Push button (AUTO/MANUAL)	two terminals	GPIO14 (internal pull-up), GND
SSD1306 OLED 128×64, I²C 0x3C	SDA, SCL, VCC, GND	GPIO21, GPIO22, 3V3, GND

GPIO34 is an ADC1 channel; ADC2 channels cannot be used while the Wi-Fi radio is active.

2.3 Sensors and Actuators

- **DHT22:** single-wire digital sensor, –40 to 80 °C (±0.5 °C), 0–100 % RH (±2 %). It can be read at most once every 2 s, which sets the sampling interval.
- **LDR module:** the photoresistor and a 10 kΩ resistor form a voltage divider. The 12-bit ADC gives the ratio r = ADC/4095, so $R_{LDR} = 10\,k\Omega \cdot r / (1 - r)$. Lux is obtained from the LDR's power law, lux = $(R_{10} \cdot 10^{\gamma} / R_{LDR})^{1/\gamma}$ with $\gamma = 0.7$ and $R_{10} = 50\,k\Omega$. Using the ratio makes the result independent of the supply voltage.
- **Relay modules:** active-HIGH; they isolate the 3.3 V logic from the switched load.
- **Buzzer:** 2 kHz tone, switched on and off every 400 ms while the alarm is active.
- **OLED:** shows readings, relay states, mode and Wi-Fi/MQTT status, or `!! HIGH TEMP !!` during an alarm.

2.4 Communication Protocol

MQTT is a lightweight publish/subscribe protocol: clients publish messages to named topics on a broker, and the broker forwards them to every subscriber. The ESP32 uses `PubSubClient` over TCP port 1883. A browser cannot open raw TCP sockets, so the dashboard uses MQTT over secure WebSockets (port 8884) on the same broker. The device registers a retained **Last Will** message (`offline`) on its status topic, so the dashboard learns when the device disconnects unexpectedly.

Table 2.2: *MQTT topics (base iot-a3/sachin-room1/)*

Topic	Direction	Content
`telemetry`	ESP32 → dashboard, every 2 s	`{"temp":26.0,"hum":55.0,"lux":50.1,"fan":false,"light":true,"mode":"auto","alert":false,...}`
`status`	ESP32 → dashboard, retained + LWT	`online` / `offline`
`alert`	ESP32 → dashboard, on events	`{"type":"high_temp","msg":"Critical room temperature!","value":42.0}`
`cmd`	dashboard → ESP32	`{"mode":"manual"}`, `{"fan":false}`, `{"light":true}`, `{"thrTemp":28}`, `{"thrLux":200}`, `{"alertTemp":45}`

3. Implementation

3.1 Firmware

The firmware (`sketch.ino`, Arduino framework) uses a non-blocking `loop()`: each task runs when its `millis()` timer expires, so MQTT, the button and the buzzer are serviced continuously.

- **Every 2 s:** read sensors → evaluate rules → publish telemetry.
- **Every 0.5 s:** redraw the OLED.
- **Continuously:** MQTT client loop, debounced button, buzzer pattern, Wi-Fi/MQTT reconnection (one attempt every 3 s, without blocking).
- **On a command:** update mode, relays or thresholds; re-evaluate; publish telemetry immediately.

3.2 Decision Logic

Table 3.1: *Threshold rules (AUTO mode; thresholds can be changed from the dashboard)*

Rule	ON condition	OFF condition
Fan (relay 1)	T ≥ 30 °C	T ≤ 29 °C (1 °C hysteresis)
Light (relay 2)	lux < 100	lux > 150 (50 lx hysteresis)
Alarm	T ≥ 40 °C → buzzer, `alert` message, fan forced on	T ≤ 39 °C → `high_temp_clear` message
Sensor fault	DHT22 read fails → `sensor` alert	sensor recovers → `sensor_ok`

Hysteresis prevents the relays from switching rapidly when a reading hovers around a threshold. Alerts are edge-triggered, so one message is sent per event. In MANUAL mode the relays follow only dashboard commands, but the alarm stays active.

Listing: sketch.ino: threshold-based decision logic (excerpt)

```
  if (sensorOk) {
    if (!alertOn && temperature >= alertTempC) {
      alertOn = true;
      publishAlert("high_temp", "Critical room temperature!", temperature);
    } else if (alertOn && temperature <= alertTempC - TEMP_HYST) {
      alertOn = false;
      publishAlert("high_temp_clear", "Temperature back to normal", temperature);
    }
  }
  if (!autoMode) return;   // MANUAL: relays only follow dashboard commands

  if (sensorOk) {
    if (temperature >= thrTempC)                  setFan(true);
    else if (temperature <= thrTempC - TEMP_HYST) setFan(false);
    if (alertOn) setFan(true);                    // safety: always cool when critical
  }
  if (lux < thrLux)                 setLight(true);
  else if (lux > thrLux + LUX_HYST) setLight(false);
```

3.3 Dashboard

`dashboard.html` is a single static page (MQTT.js and Chart.js), so it needs no server. It shows broker and device status, live readings, relay states, a trend chart (temperature and humidity on the left axis, lux on a logarithmic right axis), an alert banner and a time-stamped event log. It also provides AUTO/MANUAL selection, manual fan and light switches, and threshold inputs.

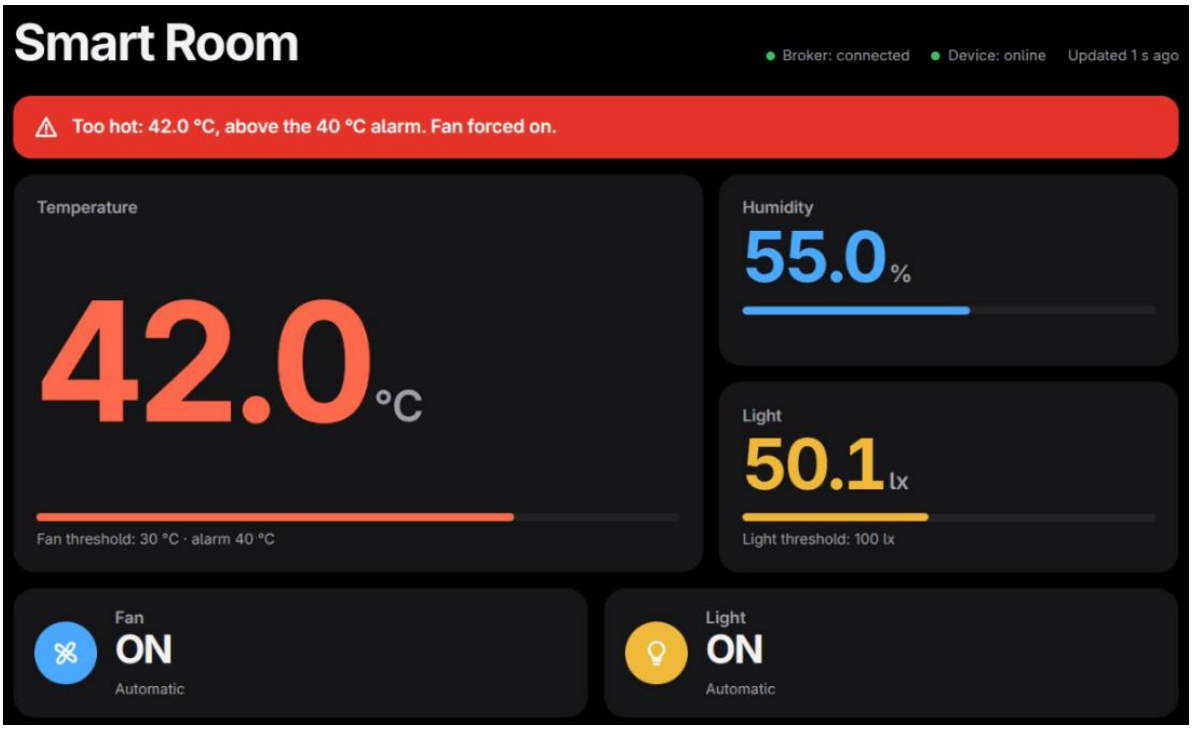


Fig. 3.1: *Web dashboard with live telemetry from the simulated ESP32 during the alarm test (temperature stepped 26 → 33 → 37 → 42 °C)*

4. Results

The complete system was run end to end: Wokwi ESP32 → `broker.hivemq.com` → dashboard in a separate browser tab. Each change below was made with the sensor's slider in Wokwi, the dashboard or the push button. The result was checked on the relays and LEDs, the OLED, the serial log and the dashboard.

Table 4.1: *Functional test results (Wokwi simulation, 03/11/2026)*

Test	Input	Observed result
Connectivity	Power-on	Wi-Fi connected (IP 10.10.0.2), MQTT connected; dashboard shows device *online*
Light measurement	LDR set to 500 lx, then 50 lx	Firmware reports 499.6 lx and 50.1 lx
Fan automation	T 26 → 32 °C	`[ACT] Fan -> ON`, relay 1 and blue LED on; dashboard logs "Fan turned ON (auto)"
Critical alarm	T → 42 °C	Buzzer on, OLED `!! HIGH TEMP !!`, dashboard alert banner + log "Critical room temperature! (42.0)"
Light automation	lux 500 → 50	Relay 2 and yellow LED on; dashboard "Light turned ON (auto)"
Remote mode change	Dashboard → MANUAL	OLED and dashboard show MANUAL
Remote actuation	Dashboard fan switch → off	Relay 1 off within one telemetry cycle; log "Fan turned OFF (manual)"
Local mode change	AUTO/MAN button	Mode returns to AUTO
Alarm clear	T → 26 °C	Buzzer off; dashboard log "Temperature back to normal (26.0)"

Issues found during testing and fixed in the firmware:

- **Lux conversion:** the first lux formula assumed the wrong divider resistor and read 905 lx for a 500 lx input. Rewriting it in terms of the ADC ratio fixed it (499.6 lx).

- **MQTT keep-alive:** the simulator sometimes runs at 3–40 % of real time, so a 15 s keep-alive (in simulated time) let the broker drop the connection. It was raised to 60 s.
- **Wi-Fi retry:** a 10 s retry that called `WiFi.disconnect()` could interrupt a slow first connection. The firmware now relies on the ESP32's auto-reconnect, with a full retry only after 30 s.

5. Limitations and Improvements

- **Simulation only:** the system was demonstrated in Wokwi rather than on physical hardware, and LEDs stand in for the fan and lamp. The firmware is hardware-ready: only the Wi-Fi credentials change.
- **Security:** the public broker has no authentication or encryption on the device side. A private broker (e.g. Mosquitto) with credentials and TLS on port 8883 should be used.
- **No history:** the dashboard keeps only about 2 minutes of live data. A time-series database (InfluxDB/Grafana or ThingSpeak) would store long-term trends.
- **Volatile settings:** thresholds changed from the dashboard reset on reboot. They could be saved in flash (NVS, `Preferences`).
- **Sensing:** the DHT22 is slow (0.5 Hz). A PIR occupancy sensor and PWM fan-speed control would save more energy than on/off control.

6. Conclusion

An IoT smart room controller was implemented on the ESP32:

- two sensors (DHT22 and LDR) are sampled every 2 s;
- the fan and light relays are controlled locally with hysteresis, and a buzzer + MQTT alert is raised at critical temperature;
- telemetry, status (with Last Will) and alerts are published over MQTT, and commands are received from the dashboard;
- the web dashboard provides live monitoring, trends, an event log and remote control.

All the functional tests in Section 4 passed in the end-to-end simulation.

Implementation. The Wokwi project (`sketch.ino`, `diagram.json`, `libraries.txt`), the dashboard and run instructions are available at https://github.com/schctl/iot_project.

References

1. Espressif Systems, "ESP32 Series Datasheet," v4.x. https://www.espressif.com/en/support/documents/technical-documents
2. Aosong Electronics, "Digital-output relative humidity & temperature sensor/module DHT22 (AM2302)," datasheet.
3. OASIS, "MQTT Version 3.1.1," OASIS Standard, 2014. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html
4. Wokwi, "Wokwi Simulator Documentation: diagram format, photoresistor sensor, ESP32 Wi-Fi." https://docs.wokwi.com
5. N. O'Leary, "PubSubClient: Arduino client for MQTT." https://pubsubclient.knolleary.net
6. HiveMQ, "Public MQTT Broker." https://www.hivemq.com/mqtt/public-mqtt-broker/
7. MQTT.js contributors, "MQTT.js: the MQTT client for Node.js and the browser." https://github.com/mqttjs/MQTT.js